DAY 7 – PYTHON EXCEPTION HANDLING AND ERROR MANAGEMENT FOR AWS DATA ENGINEERING

Today we will learn:

1. What is an Exception?
2. Syntax errors vs runtime exceptions
3. Why exception handling is important
4. try and except
5. Handling ZeroDivisionError
6. Handling ValueError
7. Handling FileNotFoundError
8. Handling multiple exceptions
9. Using else
10. Using finally
11. Raising exceptions
12. Exception handling in functions
13. Exception handling with loops
14. Validating data types
15. Handling invalid CSV records
16. Logging errors
17. Retry logic
18. Pipeline failure handling
19. Practical Data Engineering example
20. Practice programs

1. What is an Exception?

An exception is an error that happens while a Python program is running.

For example:

```
a = 10
b = 0

result = a / b
print(result)
```

Output

ZeroDivisionError: division by zero

The program stops because division by zero is not allowed.

2. Why exception handling is important

In Data Engineering, errors can happen when:

- A source file is missing.
- A CSV contains invalid values.
- A record cannot be converted to an integer.
- A network request fails.
- A service is temporarily unavailable.

Exception handling allows us to detect these problems and respond appropriately.

3. Using try and except

```
try:
    a = 10
    b = 0
    result = a / b
except ZeroDivisionError:
    print("Cannot divide by zero")
```

Output

Cannot divide by zero

Explanation: Python executes the code inside try. When a ZeroDivisionError occurs, Python executes the matching except block instead of displaying an unhandled traceback.

4. Handling invalid numbers

```
try:
    records = int("ABC")
except ValueError:
    print("Invalid record count")
```

Output

Invalid record count

int() cannot convert "ABC" into an integer.

5. Handling user input

```
try:
    records = int(input("Enter record count: "))
    print("Records:", records)
except ValueError:
    print("Please enter a valid number")
```

Example input

ABC

Output

Enter record count: ABC

Please enter a valid number

6. Handling a missing file

```
try:
    with open("missing_file.csv", "r") as file:
        data = file.read()
except FileNotFoundError:
    print("File not found")
```

Output, assuming the file does not exist:

File not found

This is useful when a pipeline expects a source file that has not arrived.

7. Handling multiple exceptions

```
try:
    records = int("ABC")
    result = 100 / records
except ValueError:
    print("Invalid number")
except ZeroDivisionError:
    print("Cannot divide by zero")
```

Output

Invalid number

The first exception is handled by the matching except block.

8. Using else

The else block runs when no exception occurs in try.

```
try:
    records = int("5000")
except ValueError:
    print("Invalid records")
else:
    print("Valid Records:", records)
```

Output

Valid Records: 5000

9. Using finally

The finally block runs whether an exception occurs or not.

```
try:
    records = int("ABC")
except ValueError:
    print("Invalid record count")
finally:
    print("Validation completed")
```

Output

Invalid record count

Validation completed

finally is useful for cleanup activities.

10. Raising an exception

We can intentionally raise an exception when data does not meet a business rule.

```
def validate_records(records):
    if records <= 0:
        raise ValueError("Record count must be greater than zero")
    print("Validation successful")

validate_records(5000)
```

Output

Validation successful

If we call validate_records(0), Python raises a ValueError with the message provided.

11. Exception handling inside a function

```
def convert_records(value):
    try:
        records = int(value)
        return records
    except ValueError:
        return 0

print(convert_records("5000"))
print(convert_records("ABC"))
```

Output

5000

0

Here, returning zero for invalid input is a simple teaching example. In a production pipeline, invalid data should usually be reported or rejected rather than silently converted to zero.

12. Exception handling with a loop

```
records = ["100", "200", "ABC", "400"]

for record in records:
    try:
        value = int(record)
        print("Valid:", value)
    except ValueError:
        print("Invalid:", record)
```

Output

Valid: 100

Valid: 200

Invalid: ABC

Valid: 400

The loop continues processing other records even when one record is invalid.

13. Validating multiple data types

```
def validate_amount(amount):
    try:
        value = float(amount)
        if value < 0:
            print("Negative amount not allowed")
        else:
            print("Valid Amount:", value)
    except ValueError:
        print("Invalid amount")

validate_amount("2500.50")
validate_amount("ABC")
validate_amount("-100")
```

Output

14. Handling invalid CSV records